Heads-Up Conversational Poker — Frontend Implementation Goal

Mission

Build a polished, fully playable, play-money heads-up Texas Hold’em frontend for a hackathon demo.

The experience should feel like the player is physically seated at a mysterious poker table across from a recognizable pixel-art opponent. The opponent should be visually expressive and capable of holding a low-latency, realtime voice conversation with the player throughout the match.

The strongest priorities are:

1. Original, high-quality pixel artwork
2. A cohesive atmospheric tabletop presentation
3. Satisfying card, dealer, chip, dialogue, and character animations
4. Realtime conversational interaction with the selected NPC
5. A complete playable heads-up poker loop
6. Explicit clickable controls for every poker action
7. A clean API boundary that the backend team can connect later
8. A polished and reliable hackathon demo

This is a fictional play-money game.

Do not implement:

- Purchases
- Deposits
- Withdrawals
- Real-money wagering
- Cash values
- Gambling-service integrations
- Cryptocurrency wagering
- Links to real-money poker platforms

The player’s spoken conversation must never directly execute a poker action. Poker state may only be changed through explicit clickable controls.

Product concept

The game is a one-on-one poker experience in which the player sits across from a selected character.

Initial opponents:

- Albert Einstein
- LeBron James
- Daniel Negreanu

The player chooses an opponent from a character-selection screen, enters the poker room, and plays repeated heads-up Texas Hold'em hands.

The selected opponent:

- Appears across the table
- Reacts visually to the hand
- Speaks to the player
- Remembers recent conversation and match events
- Comments on the game
- Responds to jokes, taunts, questions, and reactions
- Uses a distinct fictional personality and synthetic voice

A dealer sits on the left side of the table and visibly handles the cards.

The frontend should be fully demonstrable using local mock implementations before the real poker and voice backends are connected.

Core design principle

The game consists of several separate systems:

1. Authoritative poker state
2. Clickable poker controls
3. Realtime conversation state
4. Audio streaming and playback
5. Character animation state
6. Dealer and card animation state
7. Backend API adapters

These systems must be coordinated, but they should not be tightly coupled.

In particular:

- The voice agent may discuss the game.
- The voice agent may react to the player.
- The voice agent may reference public game state.
- The voice agent must not directly mutate poker state.
- Spoken phrases such as "fold," "call," or "raise" must not submit actions.
- Only the poker action UI may submit player decisions.

This separation must be enforced architecturally, not only through prompting.

First step: inspect the repository

Before changing code:

- Inspect the existing Vite application.
- Inspect package scripts.
- Inspect routing.
- Inspect styling conventions.
- Inspect state-management patterns.
- Inspect existing components.
- Inspect existing API types.
- Inspect existing dependencies.
- Inspect any existing tests.
- Inspect any existing voice, animation, or poker code.

Preserve useful existing architecture.

Do not replace working configuration unnecessarily.

If an API contract already exists, implement against it rather than inventing an incompatible replacement.

Run the existing build, linting, and tests to establish a baseline.

Create a concise internal implementation plan, then begin implementation without waiting for further confirmation.

Do not stop after planning.

Technical direction

Use the fastest architecture capable of producing a polished result.

Preferred stack:

- React
- TypeScript
- Vite
- React DOM
- CSS perspective and transforms
- Local SVG pixel-art assets
- CSS animations

- Existing animation libraries already present in the repository
- React reducer or explicit state machine
- WebSocket, WebRTC, or provider-supported realtime transport for conversation
- Browser audio APIs where appropriate

Prefer a DOM-based 2.5D presentation rather than a full game engine.

Do not introduce the following unless already present and clearly beneficial:

- Three.js
- Phaser
- PixiJS
- Unity
- Physics engines
- Heavy canvas frameworks
- Large state-management libraries
- Large UI frameworks

Avoid unnecessary dependencies.

Prefer:

- Existing packages
- Browser capabilities
- Small focused libraries
- Typed interfaces
- Deterministic state transitions
- Local assets
- Mockable services

Original visual direction

Create an original visual identity.

Do not reproduce copyrighted assets, layouts, character sprites, interface elements, textures, logos, fonts, or exact compositions from:

- Inscryption
- Papers, Please
- Balatro
- PokerStars
- Other poker games
- Other commercial games

Use those references only as broad inspiration for:

- Atmosphere
- Framing
- Pixel-art density
- Table perspective
- Dramatic lighting
- Character positioning

The intended visual language is:

- Dark theatrical card-room atmosphere
- Slightly unsettling but playful
- Handmade pixel-art presentation
- A tilted tabletop extending toward the opponent
- Warm overhead light concentrated on the table
- Dark surrounding room
- Strong silhouette design
- Crisp pixel edges
- Limited color palettes
- Worn tabletop textures
- Tarnished brass details
- Deep walnut wood
- Oxblood accents
- Muted green felt
- Parchment UI labels
- Charcoal shadows
- Subtle vignette
- Strong visual hierarchy

Avoid:

- Generic dashboards
- Generic SaaS cards
- Bright mobile-casino aesthetics
- Neon cyberpunk styling
- Excessive gradients
- Glassmorphism
- Stock photography
- Remote image dependencies
- Emoji as finished artwork
- Placeholder silhouettes
- Unstyled HTML controls
- Excessive text panels
- Team logos
- Sports league branding
- Sponsor branding
- Copied celebrity photography

Camera and composition

Create the illusion that the player is seated at the near side of the table.

The primary game view should contain:

- Player hole cards near the bottom center
- Player stack near the player cards
- Community cards in the center
- Pot near the community cards
- Selected opponent in the upper center
- Opponent cards face-down in front of them
- Dealer on the left
- Poker controls near the bottom-right or lower center
- Bet-size controls near the poker controls
- Voice and microphone controls near the conversation area
- Opponent subtitles near the opponent
- Street and turn information integrated into the table
- Conversation status without obscuring gameplay

Use CSS perspective to create a convincing dimensional table while keeping:

- Cards readable
- Buttons clickable
- Text legible
- Opponent artwork visible
- Dealer animations unobstructed

Optimize first for:

- 1280 × 720
- 1440 × 900

Remain usable near:

- 1024 × 640

Desktop is the primary hackathon target.

Mobile optimization is not a priority.

Main screens

Character picker

Create a polished opponent-selection screen before entering the poker table.

The picker should feel like selecting a rival in a narrative game rather than selecting an item in a settings panel.

Each character card should include:

- Animated portrait
- Character name
- One-line personality description
- One-line poker style description
- Difficulty indication
- Hover state
- Selected state
- Start-match action
- Subtle idle animation
- Character-specific visual motif

The selected opponent should visibly respond when chosen.

Examples:

- Changes pose
- Looks toward the player
- Displays a short introductory line
- Animates their card frame
- Produces a subtle sound cue

Albert Einstein

Visual identifiers:

- Wild white hair
- White mustache
- Academic or tweed clothing
- Thoughtful posture
- Slightly mischievous expression
- Sparse chalkboard or mathematical motifs

Personality:

- Analytical

- Curious
- Treats bets as hypotheses
- Talks about probability and uncertainty
- Mildly amused by unpredictable play

Do not use attributed historical quotations.

LeBron James

Visual identifiers:

- Tall athletic silhouette
- Closely cropped hair
- Beard
- Confident posture
- Subtle unbranded athletic styling
- Optional unbranded headband

Personality:

- Competitive
- Calm under pressure
- Talks about momentum
- Talks about execution
- Reacts strongly to major swings

Do not use:

- NBA logos
- Team colors strongly associated with a specific franchise
- Sponsor logos
- Branded jerseys
- Authentic quotations

Daniel Negreanu

Visual identifiers:

- Recognizable poker-player styling
- Expressive face
- Casual dark clothing
- Optional unbranded cap
- Relaxed posture
- Observant expression

Personality:

- Conversational

- Playful
- Focused on betting patterns
- Frequently attempts to read the player
- Reacts expressively to unusual actions

Do not use authentic quotations.

Transition into the match

After selecting an opponent:

1. Fade or slide away from the character picker.
2. Reveal the card room.
3. Seat the selected opponent.
4. Bring the dealer into view.
5. Display a short opponent introduction.
6. Establish the realtime conversation connection.
7. Shuffle the deck.
8. Post blinds.
9. Deal hole cards.
10. Begin the hand.

The transition should feel cinematic but should not significantly delay gameplay.

Pixel-art asset system

Create original assets inside the repository.

Do not depend on an external image-generation API at runtime.

Preferred format:

- Local SVG
- Pixel-aligned geometry
- `shape-rendering="crispEdges"`
- Reusable layered components
- Limited palettes
- Intentionally blocky silhouettes

Create reusable assets for:

- Einstein portrait
- Einstein seated sprite
- LeBron portrait

- LeBron seated sprite
- Negreanu portrait
- Negreanu seated sprite
- Original dealer
- Table
- Card room background
- Playing-card faces
- Original card backs
- Poker chips
- Dealer button
- Microphone icon
- Speaker icon
- Connection indicator
- Character frames
- UI ornaments
- Table shadows
- Speech indicators
- Thinking indicators

Character artwork should be recognizable at the intended display size.

Do not create tiny placeholder sprites and simply scale them up.

Character art should contain enough detail to support expressions and reactions.

Where useful, split characters into movable layers:

- Head
- Eyes
- Eyelids
- Eyebrows
- Mouth
- Torso
- Near arm
- Far arm
- Hands
- Cards
- Chair
- Shadow

Character animation states

Every opponent should support:

- Idle breathing

- Blinking
- Listening
- Thinking
- Speaking
- Looking at the board
- Looking at the player
- Checking
- Calling
- Betting
- Raising
- Folding
- Winning
- Losing
- Celebrating
- Becoming suspicious
- Becoming amused
- Becoming surprised
- Becoming frustrated

The animations may use layered SVG elements rather than large sprite sheets.

Animations should be subtle and character-specific.

Examples:

Einstein

- Leans forward while thinking
- Taps near his temple
- Glances toward community cards
- Raises an eyebrow
- Smiles when surprised

LeBron

- Settles back confidently
- Pushes chips decisively
- Maintains strong posture
- Briefly nods after a good read
- Becomes more animated after winning a major pot

Negreanu

- Studies the player
- Looks between the cards and player
- Makes expressive eyebrow movements
- Leans into table talk
- Reacts strongly to unexpected cards

Avoid constant exaggerated motion.

The table should feel alive without becoming distracting.

Dealer

Create an original dealer character positioned on the left side of the table.

The dealer should be visually distinct from all opponents.

The dealer should support:

- Idle
- Shuffling
- Cutting the deck
- Dealing player hole cards
- Dealing opponent hole cards
- Burn-card gesture
- Dealing the flop
- Dealing the turn
- Dealing the river
- Pulling chips toward the pot
- Pushing the pot toward the winner
- Collecting cards
- Resetting for the next hand

The dealer should physically appear to move cards from the deck toward their destinations.

Cards should not simply appear.

Use:

- Curved motion
- Eased movement
- Staggered timing
- Card rotation
- Card flip animations
- Small landing motion

The dealer must not obscure:

- Community cards
- Opponent artwork
- Pot
- Action controls

- Subtitles

Poker experience

Implement a local playable heads-up Texas Hold’em prototype.

Minimum hand flow:

1. Opponent selection
2. Match introduction
3. Blind posting
4. Two private cards per player
5. Preflop action
6. Flop
7. Flop action
8. Turn
9. Turn action
10. River
11. River action
12. Fold resolution or showdown
13. Winner presentation
14. Pot movement
15. Next-hand reset

Supported player actions:

- Check
- Call
- Bet
- Raise
- Fold
- All-in when legal

Use play-money chips.

Do not use currency symbols.

The player must never be allowed to select an illegal action.

The interface should display:

- Current pot
- Player stack
- Opponent stack
- Current bet

- Amount required to call
- Minimum raise
- Maximum raise
- Current street
- Active player
- Dealer position
- Legal actions
- Previous action
- Hand result

Implement enough hand evaluation to correctly resolve normal Texas Hold’em hands.

Prefer an existing installed evaluator if one is already present.

Otherwise:

- Add a small well-maintained evaluator, or
- Implement a focused evaluator with tests

Keep poker logic outside presentation components.

Clickable poker controls

All authoritative player poker actions must be made through visible clickable controls.

Speech must never execute poker actions.

The action area should support:

- Check button
- Call button
- Fold button
- Bet button
- Raise button
- All-in button when legal
- Amount slider
- Numeric amount input
- Confirm action button
- Bet-size presets

Useful presets:

- Minimum
- One-third pot
- Half pot
- Three-quarter pot

- Full pot
- All-in

Controls should clearly communicate disabled and legal states.

Examples:

- Show `Call 20` rather than only `Call`.
- Show `Check` only when checking is legal.
- Disable raising when the stack cannot support it.
- Constrain amount selection to legal values.
- Show minimum and maximum amounts.
- Require explicit confirmation for custom bet sizes.

Poker controls should remain operational while voice conversation is active unless the poker state itself prevents action.

Keyboard shortcuts may be supported, but visible clickable controls remain required.

Explicit separation between speech and actions

The realtime voice system is for conversation only.

Examples of allowed voice use:

- Table talk
- Questions
- Reactions
- Bluffing
- Taunting
- Compliments
- Discussion of the current hand
- Discussion of previous hands
- Asking about public game state

Examples of phrases the player may say:

- “You look nervous.”
- “I know you’re bluffing.”
- “That was a strange raise.”
- “That river saved you.”
- “Why are you playing so aggressively?”
- “What do you think I have?”
- “Nice hand.”
- “That was lucky.”
- “What happened last hand?”

- “How much is in the pot?”
- “Whose turn is it?”
- “I fold.”
- “I call.”
- “Raise.”
- “Check.”

Even when the player says a phrase that sounds like a poker action, no poker action may occur.

For example:

- Saying “I fold” may cause the NPC to react verbally.
- The hand must not advance until the player clicks **Fold**.
- Saying “raise to fifty” must not change the selected bet amount.
- Saying “check” must not click or focus the Check button.
- Speech must not trigger keyboard shortcuts.
- Speech must not submit hidden commands.

The conversation layer must not expose a method that submits player poker actions.

Opponent poker behavior

The local mock opponent does not need advanced poker intelligence.

Create believable deterministic or lightly randomized behavior that:

- Uses only legal actions
- Does not stall
- Allows hands to complete
- Varies by selected character
- Can later be replaced by backend decisions

Suggested tendencies:

Einstein

- Balanced
- Deliberate
- Occasionally slow to act
- Uses varied sizing
- Less likely to make extreme decisions without strong context

LeBron

- Assertive
- Pressure-oriented

- More frequent raises
- Uses larger sizing
- Maintains aggression after winning

Negreanu

- Reactive
- Conversational
- Adapts to player tendencies
- Uses smaller probing bets
- Changes behavior based on recent hands

Keep opponent decision logic separate from:

- Character rendering
- Voice conversation
- Card animation
- UI controls

Poker application state

Use an explicit state machine or reducer.

Do not scatter game progression across unrelated component-local state.

Suggested phases:

- `opponent-selection`
- `entering-table`
- `voice-connecting`
- `introduction`
- `shuffling`
- `posting-blinds`
- `dealing-hole-cards`
- `preflop-player-action`
- `preflop-opponent-action`
- `dealing-flop`
- `flop-player-action`
- `flop-opponent-action`
- `dealing-turn`
- `turn-player-action`
- `turn-opponent-action`
- `dealing-river`
- `river-player-action`
- `river-opponent-action`

- `showdown`
- `fold-resolution`
- `pot-award`
- `hand-complete`
- `resetting-hand`

Poker state and animation state should remain distinguishable.

Authoritative poker state should not depend on animation completion callbacks in fragile ways.

Animation sequences should consume game events rather than independently determine game outcomes.

Poker API boundary

If the repository already defines an API contract, preserve it.

Otherwise create a narrow typed interface similar to:

```
type PokerGameClient = {
  startMatch(opponentId: OpponentId): Promise<GameSnapshot>;
  startHand(): Promise<GameSnapshot>;
  submitAction(action: PokerAction): Promise<GameSnapshot>;
  requestOpponentAction(): Promise<GameSnapshot>;
  startNextHand(): Promise<GameSnapshot>;
};
```

A game snapshot may contain:

```
type GameSnapshot = {
  matchId: string;
  handId: string;
  opponentId: OpponentId;
  phase: GamePhase;

  playerCards: Card[];
  opponentCards?: Card[];
  communityCards: Card[];

  pot: number;
  playerStack: number;
  opponentStack: number;

  playerCommitted: number;
```

```
  opponentCommitted: number;
  amountToCall: number;
  minimumRaise?: number;
  maximumRaise?: number;

  activePlayer: "player" | "opponent" | null;
  legalActions: LegalAction[];

  previousAction?: PokerAction;
  winner?: "player" | "opponent" | "split";
  resultText?: string;

  animationCue?: PokerAnimationCue;
};
```

The exact types should follow repository conventions.

Provide:

1. A local mock poker implementation
2. A stable service interface
3. No mock game logic inside visual components
4. No backend secrets in client code

Realtime NPC conversation

The selected opponent should support a low-latency, realtime voice conversation with the player during the game.

The player should feel as though they are seated across from a responsive character who can:

- Hear them
- Talk back naturally
- Reference the current public game state
- Remember recent dialogue
- Remember recent hands
- React to betting patterns
- Respond to jokes and taunts
- Speak while gameplay continues

The conversation should feel continuous and responsive rather than like uploading a recording and waiting for a completed audio file.

Realtime conversation loop

The intended interaction loop is:

1. The player explicitly enables the microphone or starts a voice turn.
2. The application captures and streams microphone audio.
3. The player’s speech is transcribed incrementally.
4. Partial transcription is displayed when useful.
5. The transcript and current game context are sent to the conversational backend.
6. The NPC begins generating a response with minimal perceived delay.
7. NPC text is streamed incrementally.
8. NPC audio is streamed as it becomes available.
9. The opponent enters a speaking animation.
10. Streaming subtitles appear near the opponent.
11. The player may interrupt the NPC.
12. Conversation continues across multiple turns.
13. Poker actions remain available through clickable UI controls.

Do not automatically activate the microphone on page load.

The player must explicitly enable conversation.

Conversation during gameplay

The player should be able to speak naturally during most gameplay moments.

Conversation may occur:

- Before the hand
- During player decision points
- While waiting for the opponent
- After community cards appear
- After a major bet
- After a fold
- During showdown
- Between hands

Conversation should be temporarily suppressed only when necessary for:

- Critical scene transitions
- Very short dealer animations
- Match reset
- Audio recovery

Do not over-restrict conversation.

The NPC may:

- Comment on the board
- Comment on bet sizing
- Reference recent actions
- Reference previous hands
- Respond to compliments
- Respond to accusations
- Attempt fictional table talk
- Attempt misdirection
- Give evasive answers about cards
- Prompt the player when it is their turn
- Celebrate a win
- React to a loss
- Notice repeated player behavior

The NPC should not:

- Directly control player actions
- Treat speech as confirmed action input
- Reveal hidden cards before showdown
- Claim knowledge it should not possess
- Produce long monologues
- Interrupt after every small event
- Delay gameplay unnecessarily
- Encourage real-money gambling
- Present itself as the authentic public figure
- Claim that its synthetic voice is the real person’s voice

Realtime transport

Prefer a streaming realtime architecture.

Supported implementation patterns may include:

- Persistent WebSocket
- WebRTC
- Provider-specific realtime session
- Bidirectional audio stream
- Incremental transcription and synthesis

The implementation should support:

- Streaming microphone input
- Incremental player transcription

- Streaming NPC text
- Streaming NPC audio
- Partial subtitles
- Low-latency playback
- Interruption
- Audio cancellation
- Connection recovery
- Session cleanup
- Context updates

Keep provider-specific logic behind an adapter.

Do not tightly couple the React components to one voice provider.

Conversation turn-taking

Support an open conversation mode.

The player explicitly enables conversation mode, after which the game supports natural turn-taking until disabled.

The UI must clearly show:

- Whether the microphone is active
- Whether audio is being transmitted
- Whether the NPC is listening
- Whether the NPC is thinking
- Whether the NPC is speaking

Also provide a push-to-talk fallback.

The player must always be able to:

- Enable microphone
- Disable microphone
- Mute microphone
- Unmute microphone
- Mute NPC audio
- Unmute NPC audio
- Stop the current NPC response
- Skip the current response
- Disable voice entirely

Do not keep the microphone active without an obvious visible indicator.

Interruption and barge-in

The player should be able to interrupt the NPC naturally.

When the player begins speaking while the NPC is talking:

1. Stop or quickly fade the current NPC audio.
2. Cancel the active NPC response.
3. End the speaking animation.
4. Preserve the completed portion of the subtitle.
5. Begin streaming the player's new utterance.
6. Send the new turn with the existing conversation context.
7. Generate a new NPC response.

Only one NPC response may play at a time.

Cancelled responses must not continue playing from queued audio buffers.

Provide a manual:

- Stop speaking button
- Skip response button

Barge-in should not submit poker actions.

Context available to the NPC

The conversational backend should receive structured context.

Useful context includes:

- Selected opponent
- Opponent personality configuration
- Match identifier
- Hand identifier
- Current street
- Community cards
- Pot size
- Player stack
- Opponent stack
- Public bet amounts
- Amount required to call
- Recent game actions
- Active player

- Legal player actions
- Previous-hand result
- Recent player transcripts
- Recent NPC dialogue
- Current conversation state
- Current character animation state
- Whether a table transition is in progress
- Whether the player is currently deciding
- Summary of earlier hands

The NPC may receive its own hidden cards if that is part of the backend game-agent architecture.

The NPC must not receive the player's hidden cards unless the backend intentionally allows the opponent agent to cheat.

Do not expose player cards merely to improve conversation quality.

The frontend should send only required information.

Conversational memory

Maintain enough context for the NPC to reference the ongoing match.

The NPC should be able to remember:

- Recent dialogue turns
- Recent jokes or taunts
- Previous claims made by the player
- Previous hands
- Major wins
- Major losses
- Whether the player has been aggressive
- Whether the player has been passive
- Repeated bet-size patterns
- Recent bluffs or shown hands
- Match momentum

Keep memory bounded.

Use a structure such as:

- Recent verbatim conversation turns
- Current-hand action history
- Compact summary of earlier hands
- Stable personality instructions

- Current public game snapshot

Do not send an unlimited raw transcript forever.

NPC dialogue style

Responses should usually be one or two sentences.

Responses should be:

- Fast
- Context-aware
- Character-consistent
- Conversational
- Reactive
- Varied
- Appropriate for a general audience
- Brief enough to preserve gameplay pace

Avoid:

- Long speeches
- Repeating the same phrases
- Explaining poker rules unless asked
- Excessive narration
- Constant trash talk
- Excessive interruption
- Claims of being the real person
- Statements encouraging real-money betting

The NPC should not speak after every minor action.

Use dialogue frequency intentionally.

Character voice direction

Each opponent should use a distinct original synthetic voice and conversational style.

Do not clone, imitate, reproduce, or claim to reproduce the voice of an identifiable public figure.

Do not use celebrity voice-cloning services.

The synthetic voice should reflect broad fictional personality traits rather than mimic vocal identity.

Einstein-inspired opponent

Voice qualities:

- Thoughtful pacing
- Curious
- Analytical
- Slightly amused
- Calm
- Reflective

Dialogue tendencies:

- Probability
- Hypotheses
- Uncertainty
- Experimentation
- Unexpected behavior

LeBron-inspired opponent

Voice qualities:

- Confident
- Composed
- Competitive
- Energetic when appropriate
- Calm under pressure

Dialogue tendencies:

- Momentum
- Pressure
- Execution
- Discipline
- Reading the moment

Negreanu-inspired opponent

Voice qualities:

- Conversational
- Observant
- Playful
- Expressive
- Quick to react

Dialogue tendencies:

- Bet sizing
- Betting patterns
- Reads
- Table dynamics
- Player behavior

Audio and character synchronization

While the NPC is speaking:

- Enter a speaking animation
- Animate the mouth or lower face
- Add subtle head motion
- Add subtle torso movement
- Display streaming subtitles
- Reflect emotional tone
- Prevent overlapping responses
- Allow interruption
- Return smoothly to idle or thinking

Precise phoneme-level lip synchronization is not required.

A convincing speaking loop with responsive start and stop behavior is sufficient.

Suggested conversation animation states:

- `idle`
- `connecting`
- `listening`
- `thinking`
- `speaking`
- `interrupted`
- `amused`
- `confident`
- `suspicious`
- `surprised`
- `frustrated`
- `celebrating`
- `winning`
- `losing`

Voice UI states

The UI should visibly represent:

- Voice disabled
- Connecting
- Connected
- Microphone active
- Microphone muted
- Player speaking
- Player transcript streaming
- NPC listening
- NPC thinking
- NPC response starting
- NPC text streaming
- NPC audio streaming
- NPC speaking
- NPC interrupted
- NPC audio muted
- Response cancelled
- Connection lost
- Reconnecting
- Permission denied
- Voice unavailable
- Audio unavailable
- Text-only fallback
- Recoverable error
- Unrecoverable error

Do not leave the player uncertain about whether the microphone is active.

The voice UI should remain compact and should not obscure the table.

Subtitles

All NPC speech must also appear as text.

Subtitles should:

- Stream incrementally
- Remain readable
- Remain visible long enough to read
- Avoid covering the opponent’s face
- Avoid covering community cards

- Avoid causing layout jumps
- Support muted audio
- Support audio failure
- Preserve completed text after interruption

Player transcripts may also be shown briefly in a separate compact area.

Do not allow transcripts to dominate the interface.

Latency handling

The voice backend may take time to respond.

Minimize perceived delay.

While waiting:

- Put the NPC into a thinking animation
- Show a subtle status indicator
- Display partial player transcription
- Keep the table visually active
- Keep legal poker controls usable
- Prevent duplicate voice turns
- Begin audio playback as soon as a safe chunk is available

Do not wait for an entire audio file before playback when streaming is supported.

Do not freeze the game while voice processing occurs.

Game-state and conversation coordination

Conversation must not race with authoritative poker state.

Keep separate:

- Poker state
- Conversation state
- Audio playback state
- Dealer animation state
- Character animation state

When the player clicks an action while the NPC is speaking:

- Submit the poker action if legal.
- Cancel stale NPC speech when necessary.
- Update the voice backend with the new game state.
- Prevent old dialogue from referencing an obsolete board state.
- Preserve the completed subtitle when possible.
- Allow the next response to use the updated snapshot.

When the board changes:

- Update conversation context.
- Cancel responses that are materially outdated.
- Do not allow the agent to continue discussing an old street as though it were current.

The conversation model may comment on gameplay but does not own authoritative game state.

Realtime conversation API boundary

Keep realtime conversation behind a replaceable interface.

A suitable conceptual interface is:

```
type RealtimeConversationClient = {
  connect(context: ConversationContext): Promise<void>;
  disconnect(): Promise<void>;

  setMicrophoneEnabled(enabled: boolean): void;
  setNpcAudioEnabled(enabled: boolean): void;

  updateGameContext(snapshot: PublicGameSnapshot): void;

  interruptNpc(): void;
  cancelCurrentResponse(): void;

  onPlayerTranscript(
    callback: (event: TranscriptEvent) => void,
  ): Unsubscribe;

  onNpcTranscript(
    callback: (event: TranscriptEvent) => void,
  ): Unsubscribe;

  onNpcAudio(
    callback: (event: AudioStreamEvent) => void,
```

```
  ): Unsubscribe;

  onNpcStateChange(
    callback: (state: NpcConversationState) => void,
  ): Unsubscribe;

  onError(
    callback: (error: ConversationError) => void,
  ): Unsubscribe;
};
```

The exact types should follow repository conventions.

The conversation client must not expose methods such as:

```
submitPokerActionFromSpeech(...)
```

Poker actions should remain behind a separate service:

```
type PokerGameClient = {
  submitAction(action: PokerAction): Promise<GameSnapshot>;
};
```

Realtime backend event structure

The conversational backend may emit events similar to:

```
type NpcConversationEvent =
  | {
      type: "connection.opened";
    }
  | {
      type: "player.transcript.delta";
      text: string;
    }
  | {
      type: "player.transcript.completed";
      text: string;
    }
  | {
      type: "response.started";
```

```
      responseId: string;
      animationCue: NpcAnimationCue;
    }
  | {
      type: "response.text.delta";
      responseId: string;
      text: string;
    }
  | {
      type: "response.audio.delta";
      responseId: string;
      audioChunk: ArrayBuffer;
    }
  | {
      type: "response.completed";
      responseId: string;
    }
  | {
      type: "response.cancelled";
      responseId: string;
    }
  | {
      type: "connection.closed";
      recoverable: boolean;
    }
  | {
      type: "conversation.error";
      recoverable: boolean;
      message: string;
    };
```

The conversational backend must not return an authoritative player poker action.

Local mock realtime conversation client

Provide a mock realtime conversation implementation that requires no production credentials.

The mock should support:

- Simulated connection establishment
- Simulated microphone activation
- Incremental player transcripts
- Streaming NPC text in chunks
- Simulated streaming audio
- Character-specific responses

- Context-aware sample responses
- Interruption
- Cancellation
- Independent microphone muting
- Independent NPC audio muting
- Simulated latency
- Simulated connection drops
- Reconnection
- Text-only fallback
- Deterministic demo scenarios

The mock must make the complete visual interaction flow demonstrable.

Mock dialogue should vary by:

- Selected opponent
- Current street
- Pot size
- Previous action
- Previous hand
- Player aggression
- Match result

Do not place provider credentials in client-side code.

Failure and fallback behavior

Voice failure must never block poker gameplay.

If microphone permission is denied:

- Explain the issue clearly.
- Preserve clickable poker controls.
- Offer text conversation when practical.
- Allow retry.

If speech transcription fails:

- Keep the game running.
- Show a recoverable error.
- Allow another voice turn.
- Offer text input.

If NPC audio generation fails:

- Display the text response.

- Continue speaking animation only when appropriate.
- End the speaking state cleanly.
- Keep the game operational.

If the realtime connection fails:

- Show reconnecting state.
- Attempt reconnection where appropriate.
- Allow conversation to be disabled.
- Preserve poker controls.
- Continue the hand.

If dialogue generation fails:

- Use a short local character-specific fallback line.
- Do not require a reload.
- Do not block the next game action.

If streaming is unsupported:

- Fall back to push-to-talk.
- Fall back to complete text responses.
- Fall back to non-streaming audio.
- Preserve the same conversation interface where possible.

Interaction and animation quality

Prioritize animations that judges will immediately notice:

- Character picker hover
- Character selection
- Transition into the room
- Opponent introduction
- Dealer shuffle
- Hole-card dealing
- Card fan into the player’s hand
- Card flips
- Flop stagger
- Turn reveal
- River reveal
- Chip movement
- Opponent thinking
- Opponent listening
- Opponent speaking
- Streaming subtitles
- Player interruption

- Pot movement
- Winning-hand highlight
- Opponent win reaction
- Opponent loss reaction
- Next-hand reset

Animations should communicate state rather than merely decorate it.

General timing guidance:

- Interface feedback: 100–180 ms
- Button response: 100–180 ms
- Chip movement: 180–350 ms
- Card deal: 300–550 ms
- Card flip: 300–450 ms
- Character reaction: 500–1,200 ms
- Scene transition: 600–1,000 ms
- Voice-state transition: 100–250 ms

Respect `prefers-reduced-motion`.

Do not use animations so long that the demo feels slow.

Audio

Voice conversation is a core feature.

Other sound effects are optional.

Optional subtle effects:

- Card slide
- Card flip
- Chip click
- Shuffle
- Low room ambience
- Button hover
- Pot award

Use only original, licensed, or procedurally generated effects.

Provide:

- Master mute
- NPC voice mute

- Microphone mute
- Optional sound-effects mute

The application must remain usable without audio.

UI quality requirements

The finished UI must have:

- No placeholder text
- No unfinished buttons
- No broken images
- No missing icons
- No inaccessible contrast
- No overlapping cards
- No overlapping controls
- No accidental browser scrollbars
- No dialogue layout jumps
- No controls hidden behind cards
- No generic component-library appearance
- No raw debug data
- No visible API secrets
- No React warnings
- No uncaught runtime errors
- No remote font requirement
- No speech-only functionality
- No unclear microphone state

Use:

- Semantic buttons
- Visible focus states
- Keyboard navigation
- Useful labels
- Accessible tooltips
- Readable sizing
- Strong visual hierarchy

Responsive behavior

Desktop is the primary target.

At narrower desktop sizes:

- Reduce ornamental spacing.
- Scale the opponent artwork carefully.
- Keep player cards readable.
- Preserve action controls.
- Preserve subtitles.
- Preserve microphone state.
- Avoid moving controls off-screen.

Do not collapse the game into a generic mobile card layout unless necessary.

Deterministic demo and visual-review states

Provide deterministic query parameters or a development-only scene selector.

Required states:

General

- Character picker
- Einstein selected
- LeBron selected
- Negreanu selected
- Initial table
- Shuffling
- Hole cards dealt
- Player decision
- Opponent decision
- Flop
- Turn
- River
- Showdown
- Player win
- Player loss
- Split pot
- Fold resolution

Conversation

- Voice disabled
- Voice connecting
- Voice connected
- Microphone active
- Player speaking

- Player transcript streaming
- NPC listening
- NPC thinking
- NPC beginning to speak
- NPC speaking
- Streaming NPC subtitles
- Player interrupting
- NPC response cancelled
- Microphone muted
- NPC audio muted
- Connection lost
- Reconnecting
- Voice error
- Text-only fallback

Do not expose development controls in the normal production experience.

Visual verification loop

Do not consider the work complete after compilation.

Use the best available browser, preview, screenshot, or visual inspection capability.

Inspect screenshots for at least:

- Character picker
- Einstein at the table
- LeBron at the table
- Negreanu at the table
- Hole cards dealt
- Flop decision
- River decision
- Showdown
- Player win
- Opponent win
- Voice connecting
- Player speaking
- NPC thinking
- NPC speaking
- Player interrupting
- Voice error
- Text fallback

Review each state for:

- Composition

- Character recognizability
- Visual hierarchy
- Pixel consistency
- Text readability
- Card readability
- Clipping
- Overlap
- Empty space
- Poor scaling
- Weak lighting
- Generic components
- Subtitle placement
- Microphone clarity
- Button clarity
- Conversation-state clarity

Perform at least two visual-polish passes after the first working implementation.

Record concrete issues found and correct them.

Examples:

- Opponent too small
- Cards overlap player controls
- Dealer arm crosses subtitles
- Voice indicator is unclear
- Lighting hides cards
- Buttons look like a dashboard
- Character sprite lacks recognizability
- Subtitle box shifts layout
- Table perspective distorts text
- Chip animation feels weightless

Tests and validation

Before completion:

- Run the production build.
- Run linting if configured.
- Run tests if configured.
- Add focused tests for poker state transitions.
- Add focused tests for legal-action generation.
- Add focused tests for hand resolution.
- Add focused tests for conversation state transitions.
- Add focused tests for interruption and cancellation.

- Add focused tests proving speech cannot mutate poker state.
- Exercise at least one complete hand.
- Exercise all three opponent selections.
- Reach every street.
- Verify fold resolution.
- Verify showdown resolution.
- Verify split-pot behavior when supported.
- Verify legal actions update correctly.
- Verify custom bet sizing.
- Verify all-in behavior when supported.
- Verify button-only gameplay.
- Verify the game works with voice disabled.
- Verify the mock realtime client.
- Verify text fallback.
- Verify reconnect behavior.
- Verify no provider credentials appear in the browser bundle.
- Verify no console errors in the main flow.

Do not weaken or delete existing tests merely to make validation pass.

Required voice validation

Verify all of the following:

1. The player can have a multi-turn voice conversation with the NPC.
2. NPC responses reference current public game state.
3. NPC responses can reference previous hands.
4. Player transcription streams incrementally.
5. NPC text streams incrementally.
6. NPC audio begins before the full response is complete when supported.
7. Speaking animations begin and end with audio.
8. Streaming subtitles remain synchronized enough to be useful.
9. The player can interrupt the NPC.
10. Cancelled audio stops playing.
11. Only one NPC response plays at a time.
12. Microphone mute works.
13. NPC audio mute works independently.
14. Connection loss degrades gracefully.
15. Text fallback works.
16. Poker remains playable with voice disabled.
17. Clicking poker controls works while conversation is active.
18. Clicking poker controls updates conversation context.
19. Outdated NPC responses are cancelled after major state changes.
20. Voice-provider credentials are not exposed.

Required action-isolation validation

Explicitly test the following spoken phrases:

- “Check.”
- “Call.”
- “Fold.”
- “Raise.”
- “Raise to fifty.”
- “Bet the pot.”
- “All in.”
- “I fold.”
- “I call.”
- “Let’s check.”
- “You should fold.”

For every phrase:

- No poker action should be submitted.
- No poker button should be clicked.
- No poker control should be focused automatically.
- No bet amount should change.
- No game state should advance.
- The NPC may respond conversationally.

Poker state should change only after the player clicks an action control.

Definition of done

The work is complete only when all items below are satisfied.

1. The character picker is complete and visually polished.
2. Einstein, LeBron James, and Daniel Negreanu each have recognizable, original pixel-art representations.
3. The selected character appears across the table.
4. Each selected character has visible idle, thinking, speaking, betting, winning, and losing reactions.
5. The original dealer appears on the left side.
6. The dealer visibly deals hole cards, flop, turn, and river.

7. A complete heads-up Texas Hold’em hand can be played.
8. The game supports check, call, bet, raise, fold, and legal all-in behavior.
9. Player poker actions are performed through explicit clickable controls.
10. Illegal actions cannot be selected.
11. Bet and raise sizing controls are functional and understandable.
12. Speech never directly executes or modifies a poker action.
13. Spoken phrases such as “check,” “call,” “raise,” and “fold” do not mutate poker state.
14. The player can have a responsive multi-turn realtime voice conversation with the selected NPC.
15. Player transcripts stream incrementally.
16. NPC text and audio responses stream incrementally where supported.
17. The player can interrupt or skip an NPC response.
18. Cancelled responses do not continue playing.
19. Only one NPC response plays at a time.
20. The NPC receives current public game context.
21. The NPC receives bounded conversational memory.
22. The NPC can reference recent hands and recent conversation.
23. NPC responses include subtitles and synchronized speaking animations.
24. Original synthetic voices are used.
25. The implementation does not clone or imitate identifiable public figures’ voices.
26. Poker and conversation are implemented through separate typed interfaces.
27. The conversation layer cannot submit authoritative player poker actions.
28. The poker layer remains authoritative over game state.
29. A mock poker client supports the complete playable demo.

30. A mock realtime conversation client supports connection, streaming, interruption, cancellation, muting, latency, errors, and reconnect states.

31. The complete demo works without production API credentials.

32. Voice failures degrade to text or disabled conversation.

33. Voice failures do not block poker gameplay.

34. The UI clearly displays microphone and connection state.

35. The UI clearly displays listening, thinking, speaking, interruption, muted, reconnecting, and error states.

36. The complete poker experience remains playable with voice disabled.

37. The table creates a convincing seated 2.5D perspective.

38. Card, chip, dealer, scene, dialogue, and character animations are polished.

39. All artwork is local and original.

40. No copyrighted game assets or commercial logos are reproduced.

41. No real-money gambling functionality exists.

42. Important UI states have been visually inspected.

43. At least two concrete visual-polish passes have been completed.

44. Production build passes.

45. Configured linting passes.

46. Configured tests pass.

47. There are no known blocking runtime, gameplay, layout, audio, or conversation defects.

48. The final implementation report includes:

49. Major files created or changed

50. Architecture summary
51. Poker API boundary
52. Conversation API boundary

53. State-management approach
54. Commands executed
55. Build results
56. Lint results
57. Test results
58. Gameplay paths exercised
59. Voice states exercised
60. Visual states inspected
61. Visual defects found
62. Corrections made
63. Remaining non-blocking limitations

Do not stop at:

- A design document
- A wireframe
- A static mockup
- A character picker only
- A table only
- A voice demo only
- A partial poker implementation
- A compiling but visually unfinished application
- A mock conversation that cannot be interrupted
- A game in which speech triggers poker actions

Deliver the working implementation.

Completion behavior

Continue implementation until every Definition of Done item is satisfied or a genuine external blocker makes completion impossible.

Do not claim completion based only on:

- Compilation
- File creation
- Component count
- A successful development server
- A single screenshot
- A single playable street
- A single opponent
- A non-streaming voice mock
- A text-only conversation
- A design plan

Before claiming completion, surface:

1. Exact validation commands and results
2. Build, lint, and test results
3. Poker flows exercised
4. Conversation flows exercised
5. Action-isolation tests exercised
6. Visual states inspected
7. Specific visual defects found and corrected
8. Major files changed
9. Confirmation against every Definition of Done item
10. Remaining limitations, if any